ASSIGNMENT-1 & 2

Programming Solution & Demo (30%) + Viva (10%). Total 40% Weightage.

A Single Report needs to be submitted by each group. Report template will be shared later.

Submission Deadline: 5th April 2026, 9pm IST

Demonstration and Viva will be conducted post the submission.

==========

Assignment-1: Retrieval-Augmented Generation (RAG) for Domain-Specific Question Answering

Objective: The objective of this assignment is to design and implement a Retrieval-Augmented Generation (RAG) system capable of answering user queries using a domain-specific knowledge base.

Problem Statement: Large Language Models sometimes generate incorrect responses due to lack of domain knowledge. Retrieval-Augmented Generation (RAG) mitigates this limitation by retrieving relevant documents from an external knowledge source and injecting them into the prompt context before generating a response.

Students must design and implement a RAG-based application for a domain of their choice.

- Healthcare knowledge assistant
- Legal document assistant
- University course assistant
- Research paper assistant
- Financial advisory assistant
- Technical documentation assistant
- Customer support knowledge base

Required System Components

- Document ingestion and preprocessing
- Text chunking strategy (use different chunk sizes)
- Embedding generation (use different embedding models – Huggingface and others, try different embedding dimensions, etc.)
- Vector database storage and retrieval (use at least two vector database)
- Prompt engineering with retrieved context (semantic search and ranking)
- LLM-based answer generation (use different LLMs – e.g. LLaMA-3.1, Gemma-3, GPT style open models, etc.)

Deliverables

- Complete source code with modular design
- Experimental analysis report (8–10 pages)
- Comparative evaluation of at least two system configurations
- Example output demonstrating the system (screenshots, etc.)

Generative AI & LLMs

Assignment-2: Fine-Tuning a Large Language Model Using Custom Dataset

Objective: The objective of this assignment is to understand how fine-tuning improves the performance of Large Language Models for specialized tasks by adapting them to domain-specific datasets.

Problem Statement: Pretrained LLMs are trained on large general-purpose datasets and may not perform well on domain-specific tasks. Fine-tuning allows models to learn specialized behaviour by training them on custom datasets.

Create or curate a dataset and fine-tune an open-source LLM for a specific task.

- Domain-specific question answering
- Instruction-following assistant
- Code generation assistant
- Domain-specific summarization
- Customer support chatbot
- Structured data to text generation

Dataset Requirements: Construct or collect a dataset containing at least 500–2000 examples. The dataset must contain an input prompt and expected output.

Possible dataset sources:

- Synthetic dataset generated using LLMs
- Public datasets
- Custom curated datasets

Dataset must include:

- Input prompt
- Expected output

Model Options: Fine-tune models such as:

- LLaMA-based models
- Mistral
- Gemma
- GPT-style open models
- Other open-source LLMs

Recommended Approaches:

- LoRA / QLoRA
- PEFT fine-tuning
- HuggingFace Transformers

Deliverables

- Complete source code including dataset preprocessing and training pipeline
- Dataset used for fine-tuning with documentation
- Experimental report (8–10 pages) explaining methodology and results

  Report must include - Problem definition, Dataset creation methodology, Model architecture and fine-tuning method, Training configuration, Evaluation results

- Comparison between base model and fine-tuned model - metrics may include - Accuracy, BLEU / ROUGE, Human evaluation, Task-specific metrics
- Example outputs demonstrating performance improvement (screenshots, etc.)